

K L Deemed to be University

Department of Computer Science and Engineering | 2026-2027, Odd Semester
Database Systems Engineering and Distributed Backend Development (25CS1302E)

Practical Experiment 2: The Librarian's Dashboard

Name	Habeeb	Roll No.	2520090054
Course	25CS1302E	CO Mapping	CO1
Session	Practical 2	Date	

Aim

To create the Loans junction table and report each member with the book they have currently borrowed (JOIN), count books per published year (GROUP BY), implement the book-donation process as an atomic transaction with a Donation_History log, and create an index on ISBN and explain the speed difference.

Program

p2.sql (prerequisite: Experiment 1)

```
-- Experiment 2: The Librarian's Dashboard
-- Prerequisite: bookflow_db with Books and Members from Experiment 1
\c bookflow_db

-- Additional catalog data so that the reports are meaningful
INSERT INTO Books (title, isbn, published_year) VALUES
('Dune', '9780441172719', 2019),
('Atomic Habits', '9780735211292', 2018),
('Designing Data-Intensive Applications', '9781449373320', 2017),
('Project Hail Mary', '9780593135204', 2021),
('The Pragmatic Programmer', '9780135957059', 2019),
('Deep Work', '9781455586691', 2016);

INSERT INTO Members (full_name, email) VALUES
('Sneha Reddy', 'sneha.reddy@yahoo.com'),
('Arjun Nair', 'arjun.nair@kluniversity.in');

-- Task 1: Junction table Loans (Members M:N Books)
CREATE TABLE Loans (
    loan_id SERIAL PRIMARY KEY,
    member_id INT NOT NULL REFERENCES Members(member_id) ON DELETE CASCADE,
    book_id INT NOT NULL REFERENCES Books(book_id) ON DELETE CASCADE,
    loan_date DATE NOT NULL DEFAULT CURRENT_DATE,
    return_date DATE,
    CHECK (return_date IS NULL OR return_date >= loan_date)
);

-- Loans are inserted by natural keys (email, ISBN) so the script does not
-- depend on generated ID values
INSERT INTO Loans (member_id, book_id, loan_date, return_date)
SELECT m.member_id, b.book_id, v.loan_date::DATE, v.return_date::DATE
FROM (VALUES
    ('habeeb@kluniversity.in', '9780553418026', '2026-09-20', NULL),
    ('habeeb@kluniversity.in', '9780441172719', '2026-09-25', NULL),
    ('ananya.rao@gmail.com', '9780132350884', '2026-09-10', '2026-09-24'),
    ('ananya.rao@gmail.com', '9781449373320', '2026-09-28', NULL),
    ('rahul.v@outlook.com', '9780593135204', '2026-09-15', NULL),
    ('sneha.reddy@yahoo.com', '9780078022159', '2026-09-01', '2026-09-12')
) AS v(email, isbn, loan_date, return_date)
JOIN Members m ON m.email = v.email
JOIN Books b ON b.isbn = v.isbn;

SELECT * FROM Loans ORDER BY loan_id;

-- Catalog search: member name and the book title they have currently borrowed
SELECT m.full_name AS member_name,
       b.title AS book_title,
       l.loan_date
FROM Loans l
JOIN Members m ON m.member_id = l.member_id
JOIN Books b ON b.book_id = l.book_id
WHERE l.return_date IS NULL
ORDER BY m.full_name, l.loan_date;
```

```
-- Task 2: Collection statistics - total number of books published in each year
SELECT published_year,
       COUNT(book_id) AS total_books
FROM   Books
GROUP BY published_year
ORDER BY published_year;

-- Task 3: The donation transaction (ACID)
CREATE TABLE Donation_History (
    donation_id SERIAL PRIMARY KEY,
    book_id     INT         NOT NULL REFERENCES Books(book_id),
    donor_name  VARCHAR(100) NOT NULL,
    category    VARCHAR(50)  NOT NULL,
    donated_on  TIMESTAMP   NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Case A: both steps succeed, so the transaction commits
BEGIN;
INSERT INTO Books (title, isbn, published_year)
VALUES ('Introduction to Algorithms', '9780262046305', 2022);
INSERT INTO Donation_History (book_id, donor_name, category)
VALUES (currval('books_book_id_seq'), 'Rotary Club Vijayawada', 'Computer Science');
COMMIT;

-- Case B: the second step fails (category missing), so nothing is saved
BEGIN;
INSERT INTO Books (title, isbn, published_year)
VALUES ('Sapiens', '9780062316097', 2015);
INSERT INTO Donation_History (book_id, donor_name, category)
VALUES (currval('books_book_id_seq'), 'Anonymous', NULL);
ROLLBACK;

SELECT b.book_id, b.title, d.donor_name, d.category
FROM   Donation_History d JOIN Books b ON b.book_id = d.book_id;

SELECT COUNT(*) AS sapiens_rows FROM Books WHERE title = 'Sapiens';

-- Task 4: Search speed (indexing)
CREATE INDEX idx_books_isbn ON Books (isbn);
\di idx_books_isbn

-- Demonstration on a larger copy of the catalog (50,000 rows)
CREATE TABLE books_large AS
SELECT g AS book_id,
       'Book Title ' || g
       (9780000000000 + g)::TEXT AS title,
       AS isbn,
       1950 + (g % 76) AS published_year
FROM   generate_series(1, 50000) AS g;
ANALYZE books_large;

-- Before the index: ISBN search scans every row
EXPLAIN ANALYZE SELECT * FROM books_large WHERE isbn = '9780000042424';

CREATE INDEX idx_books_large_isbn ON books_large (isbn);
ANALYZE books_large;

-- After the index: ISBN search uses the B-tree index
EXPLAIN ANALYZE SELECT * FROM books_large WHERE isbn = '9780000042424';

-- Title search has no index, so it still scans every row
EXPLAIN ANALYZE SELECT * FROM books_large WHERE title = 'Book Title 42424';
```

Output

```
INSERT INTO Books (title, isbn, published_year) VALUES
('Dune', '9780441172719', 2019),
('Atomic Habits', '9780735211292', 2018),
('Designing Data-Intensive Applications', '9781449373320', 2017),
('Project Hail Mary', '9780593135204', 2021),
('The Pragmatic Programmer', '9780135957059', 2019),
('Deep Work', '9781455586691', 2016);
INSERT 0 6
INSERT INTO Members (full_name, email) VALUES
('Sneha Reddy', 'sneha.reddy@yahoo.com'),
('Arjun Nair', 'arjun.nair@kluniversity.in');
INSERT 0 2
CREATE TABLE Loans (
```

```

    loan_id    SERIAL PRIMARY KEY,
    member_id  INT    NOT NULL REFERENCES Members(member_id) ON DELETE CASCADE,
    book_id    INT    NOT NULL REFERENCES Books(book_id)    ON DELETE CASCADE,
    loan_date  DATE NOT NULL DEFAULT CURRENT_DATE,
    return_date DATE,
    CHECK (return_date IS NULL OR return_date >= loan_date)
);
CREATE TABLE
INSERT INTO Loans (member_id, book_id, loan_date, return_date)
SELECT m.member_id, b.book_id, v.loan_date::DATE, v.return_date::DATE
FROM (VALUES
    ('habeeb@kluniversity.in',      '9780553418026', '2026-09-20', NULL),
    ('habeeb@kluniversity.in',      '9780441172719', '2026-09-25', NULL),
    ('ananya.rao@gmail.com',         '9780132350884', '2026-09-10', '2026-09-24'),
    ('ananya.rao@gmail.com',         '9781449373320', '2026-09-28', NULL),
    ('rahul.v@outlook.com',          '9780593135204', '2026-09-15', NULL),
    ('sneha.reddy@yahoo.com',        '9780078022159', '2026-09-01', '2026-09-12')
) AS v(email, isbn, loan_date, return_date)
JOIN Members m ON m.email = v.email
JOIN Books  b ON b.isbn = v.isbn;
INSERT 0 6
SELECT * FROM Loans ORDER BY loan_id;
loan_id | member_id | book_id | loan_date | return_date
-----+-----+-----+-----+-----
1 | 2 | 1 | 2026-09-10 | 2026-09-24
2 | 7 | 2 | 2026-09-01 | 2026-09-12
3 | 1 | 3 | 2026-09-20 |
4 | 1 | 7 | 2026-09-25 |
5 | 2 | 9 | 2026-09-28 |
6 | 3 | 10 | 2026-09-15 |
(6 rows)

SELECT m.full_name AS member_name,
       b.title      AS book_title,
       l.loan_date
FROM   Loans l
JOIN   Members m ON m.member_id = l.member_id
JOIN   Books b ON b.book_id = l.book_id
WHERE  l.return_date IS NULL
ORDER BY m.full_name, l.loan_date;
member_name | book_title | loan_date
-----+-----+-----
Ananya Rao | Designing Data-Intensive Applications | 2026-09-28
Habeeb | The Martian | 2026-09-20
Habeeb | Dune | 2026-09-25
Rahul Verma | Project Hail Mary | 2026-09-15
(4 rows)

SELECT published_year,
       COUNT(book_id) AS total_books
FROM   Books
GROUP BY published_year
ORDER BY published_year;
published_year | total_books
-----+-----
2008 | 1
2014 | 1
2016 | 1
2017 | 1
2018 | 1
2019 | 3
2021 | 1
(7 rows)

CREATE TABLE Donation_History (
    donation_id SERIAL PRIMARY KEY,
    book_id INT NOT NULL REFERENCES Books(book_id),
    donor_name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    donated_on TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE
BEGIN;
BEGIN
INSERT INTO Books (title, isbn, published_year)
VALUES ('Introduction to Algorithms', '9780262046305', 2022);
INSERT 0 1
INSERT INTO Donation_History (book_id, donor_name, category)

```

```
VALUES (currval('books_book_id_seq'), 'Rotary Club Vijayawada', 'Computer Science');
INSERT 0 1
COMMIT;
COMMIT
BEGIN;
BEGIN
INSERT INTO Books (title, isbn, published_year)
VALUES ('Sapiens', '9780062316097', 2015);
INSERT 0 1
INSERT INTO Donation_History (book_id, donor_name, category)
VALUES (currval('books_book_id_seq'), 'Anonymous', NULL);
ERROR: null value in column "category" of relation "donation_history" violates not-null constraint
DETAIL: Failing row contains (2, 14, Anonymous, null, 2026-10-02 12:04:15.955627).
ROLLBACK;
ROLLBACK
SELECT b.book_id, b.title, d.donor_name, d.category
FROM Donation_History d JOIN Books b ON b.book_id = d.book_id;
 book_id | title | donor_name | category
-----+-----+-----+-----
      13 | Introduction to Algorithms | Rotary Club Vijayawada | Computer Science
(1 row)

SELECT COUNT(*) AS sapiens_rows FROM Books WHERE title = 'Sapiens';
 sapiens_rows
-----
          0
(1 row)

CREATE INDEX idx_books_isbn ON Books (isbn);
CREATE INDEX
      List of relations
Schema | Name | Type | Owner | Table
-----+-----+-----+-----+-----
public | idx_books_isbn | index | postgres | books
(1 row)

CREATE TABLE books_large AS
SELECT g AS book_id,
       'Book Title ' || g AS title,
       (9780000000000 + g)::TEXT AS isbn,
       1950 + (g % 76) AS published_year
FROM generate_series(1, 50000) AS g;
SELECT 50000
ANALYZE books_large;
ANALYZE
EXPLAIN ANALYZE SELECT * FROM books_large WHERE isbn = '9780000042424';
QUERY PLAN
-----
Seq Scan on books_large (cost=0.00..1073.00 rows=1 width=38) (actual time=2.977..3.463 rows=1 loops=1)
  Filter: (isbn = '9780000042424'::text)
  Rows Removed by Filter: 49999
  Planning Time: 0.077 ms
  Execution Time: 3.475 ms
(5 rows)

CREATE INDEX idx_books_large_isbn ON books_large (isbn);
CREATE INDEX
ANALYZE books_large;
ANALYZE
EXPLAIN ANALYZE SELECT * FROM books_large WHERE isbn = '9780000042424';
QUERY PLAN
-----
Index Scan using idx_books_large_isbn on books_large (cost=0.29..8.31 rows=1 width=38) (actual
time=0.019..0.020 rows=1 loops=1)
  Index Cond: (isbn = '9780000042424'::text)
  Planning Time: 0.112 ms
  Execution Time: 0.032 ms
(4 rows)

EXPLAIN ANALYZE SELECT * FROM books_large WHERE title = 'Book Title 42424';
QUERY PLAN
-----
Seq Scan on books_large (cost=0.00..1073.00 rows=1 width=38) (actual time=3.000..3.464 rows=1 loops=1)
  Filter: (title = 'Book Title 42424'::text)
  Rows Removed by Filter: 49999
  Planning Time: 0.027 ms
  Execution Time: 3.473 ms
```

(5 rows)

Result

The JOIN listed the four books currently on loan with member names, and GROUP BY reported 3 books published in 2019 and one in each other year. The donation transaction committed both rows in Case A, while in Case B the missing category made the second insert fail and ROLLBACK removed the book as well (0 rows for Sapiens), proving atomicity. Searching 50,000 rows by ISBN changed from a Seq Scan reading every row to an Index Scan through the B-tree, cutting execution time by about two orders of magnitude, while the unindexed title search still scanned all rows. ISBN search is faster because the B-tree finds one key in a few page reads ($O(\log n)$) instead of comparing every title.